

Documentação da Expansão das Macros — Grupo 7

Responsável (Fase 2): Roberto Ribeiro **Tarefa 4 (Pré-processador):** Programa NASM que exemplifica o uso de todas as macros multilinhas vistas em sala de aula. Esta documentação mostra **como cada macro foi substituída** pelo pré-processador do NASM.

1. Como o arquivo expandido foi gerado

O NASM permite rodar **apenas o pré-processador** (sem montar para objeto) com a opção `-E`. Isso emite o código-fonte já com todas as macros substituídas.

```
nasm -E macros.asm -o macros_expandido.asm
```

No Windows, com o NASM instalado em `C:\Program Files\NASM`: `powershell & "C:\Program Files\NASM\nasm.exe" -E macros.asm -o macros_expandido.asm`

- **Entrada:** `macros.asm` (código fonte do Alan — Fase 1, 206 linhas).
- **Saída:** `macros_expandido.asm` (código já pré-processado, 256 linhas).

O que muda no arquivo expandido

1. **Definições de macro somem.** Todos os blocos `%macro ... %endmacro` (linhas 47–157 do fonte) desaparecem da saída — eles existem só em tempo de pré-processamento. Restam apenas linhas em branco e marcadores `%line`.
 2. **Chamadas viram código real.** Cada *chamada* de macro no `_start` é substituída pelo corpo expandido (com os parâmetros já trocados).
 3. **Diretivas `%line N+M arquivo`.** O NASM insere essas marcações para mapear cada linha expandida de volta à linha original do fonte. Elas não geram código; servem para que mensagens de erro apontem o lugar certo. Ex.: `%line 49+1 macros.asm` indica que as linhas seguintes vieram da linha 49 (corpo da macro `print`).
 4. **Seções ganham colchetes.** `section .data` vira `[section .data]` — é a forma canônica interna que o pré-processador normaliza.
-

2. Expansão macro por macro

A seguir, cada macro do fonte, o que o pré-processador fez e o resultado no arquivo expandido.

Macro base: `print (%macro print 2)`

É a macro central — quase todas as outras a chamam. Fonte (linhas 48–54):

```
%macro print 2
    mov rax, 1
    mov rdi, 1
    mov rsi, %1
    mov rdx, %2
    syscall
%endmacro
```

Ao chamar `print msg1, len1`, o pré-processador troca `%1` → `msg1` e `%2` → `len1`:

```
%line 49+1 macros.asm
mov rax, 1
mov rdi, 1
mov rsi, msg1
mov rdx, len1
syscall
```

Repare que **todo** bloco de 5 instruções no expandido que começa em `%line 49+1` é uma expansão de `print`. Como `print` aparece dentro de quase todas as outras macros, ela é a que mais se repete na saída.

1. `%macro` básico — `print`

- **Chamada:** `print msg1, len1` (linha 162)
- **Substituição:** parâmetros posicionais `%1` e `%2` trocados por `msg1 / len1`.
- Ver bloco acima (expandido linhas 93–98).

2. `%imacro` (case-insensitive) — `printinsensitivo`

- **Fonte:** `%imacro PrintInSensitivo 2` (definida com maiúsculas).
- **Chamada:** `printinsensitivo msg2, len2` (linha 163, tudo minúsculo).
- **Por que funciona:** `%imacro` torna o nome **insensível a maiúsculas/ minúsculas**, então `printinsensitivo` casa com `PrintInSensitivo`.
- **Substituição (dupla):** `printinsensitivo` expande para `print %1,%2`, que por sua vez expande para o corpo de `print`. O expandido mostra direto o resultado final (linhas 99–104):

```
mov rax, 1
mov rdi, 1
mov rsi, msg2
mov rdx, len2
syscall
```

3. `%rep` / `%endrep` — `print_char_rep`

- **Chamada:** `print_char_rep char_rep, 3` (linha 166).
- **Substituição:** O `%rep 3` **desenrola o laço em tempo de compilação** — gera 3 cópias de `print char_rep, 1 + 1` cópia de `print newline, 1`.
- No expandido (linhas 108–131): **quatro** blocos `print` seguidos (3× imprimindo `char_rep, 1× newline`). O laço não existe em tempo de execução; já está totalmente desenrolado.

4. `%ifidn / %else / %endif` — `check_os`

- **Chamada:** `check_os linux` (linha 169).
- **Substituição:** `%ifidn %1, linux` compara o texto do parâmetro. Como `%1` é `linux`, o ramo verdadeiro é mantido e o `%else` é **descartado**.
- No expandido (linhas 135–140): aparece **só** o `print msg_linux, len_linux`. O ramo `msg_desconhecido` não existe na saída — foi eliminado pelo condicional.

5. `%if + labels locais %%` — `conditional_print`

- **Chamada:** `conditional_print 1` (linha 172).
- **Substituição:** `%if 1 > 0` é verdadeiro → corpo mantido. O label local `%%fim_macro` é **renomeado para um nome único** para evitar colisão caso a macro seja usada várias vezes.
- No expandido (linhas 144–153):

```
mov rax, 1
mov rdi, 1
mov rsi, msg_if
mov rdx, len_if
syscall
jmp ..@13.fim_macro
..@13.fim_macro:
nop
```

Note o `%%fim_macro` → `..@13.fim_macro`. O prefixo `..@13.` é o número de expansão que o NASM usa para garantir unicidade dos labels locais.

6. Parâmetro guloso `1+` — `print_guloso`

- **Chamada:** `print_guloso "6. Argumento guloso: ", "texto_1", ", texto_2", 10` (linha 175).
- **Substituição:** o `+` faz o **último parâmetro absorver todos os argumentos restantes** (com as vírgulas). Assim `%1` vira a lista inteira `"6. Argumento guloso: ", "texto_1", ", texto_2", 10`.
- No expandido (linhas 157–167): essa lista vira o conteúdo do `db`, e os labels locais viram `..@15.str`, `..@15.endstr`, `..@15.skip_data`:

```
jmp ..@15.skip_data
..@15.str: db "6. Argumento guloso: ", "texto_1", ", texto_2", 10
..@15.endstr:
..@15.skip_data:
... (print ..@15.str, ..@15.endstr - ..@15.str)
```

7. Valor default 1-2 50 — soma_com_default

- **Chamada:** soma_com_default 10 (linha 178) — só **um** argumento.
- **Substituição:** a faixa 1-2 aceita 1 ou 2 parâmetros; o 2º tem **default 50**. Como só foi passado 10, %2 assume 50.
- No expandido (linhas 171–173):

```
mov rax, 10
add rax, 50
```

O 50 foi injetado automaticamente, não veio da chamada.

8. %rotate + contador %0 (1-*) — multipush / multipop

- **Chamada:** multipush rax, rbx e multipop rax, rbx (linhas 183–184).
- **Substituição:** 1-* aceita número **ilimitado** de parâmetros; %0 é a contagem (=2). O %rep %0 desenrola, e %rotate 1 gira a lista a cada iteração, de modo que %1 aponta para o próximo argumento.
- No expandido:

```
push rax      ; %line 109
push rbx      ; %line 109 (após %rotate)
...
pop rbx       ; multipop com %rotate -1 inverte a ordem
pop rax
```

multipop usa %rotate -1 (giro reverso) para desempilhar na **ordem inversa** — rbx antes de rax — preservando a integridade da pilha.

9. Concatenação de parâmetro a texto — print_var

- **Chamadas:** print_var A e print_var B (linhas 187–188).
- **Substituição:** var%1 cola o parâmetro ao identificador. Com %1=A → varA / len_varA; com %1=B → varB / len_varB.
- No expandido: dois blocos print usando varA / len_varA e varB / len_varB.

10. Código de condição %+1 — jump_cc

- **Chamada:** jump_cc e (linha 191).
- **Substituição:** j%+1 **concatena** o prefixo j com o parâmetro e, formando a instrução je.
- No expandido (linhas 205–217):

```

cmp rax, rax
je ..@24.imprime
jmp ..@24.fim
..@24.imprime:
... (print msg_cc)
..@24.fim:

```

O `%+1` virou `je` literalmente. Labels locais → `..@24.imprime` / `..@24.fim`.

11. Sub-lista de parâmetros `{ }` — `sublist_macro`

- **Chamada:** `sublist_macro {'a', 'b', 'c'}` (linha 194).
- **Substituição:** as chaves `{ }` agrupam `'a', 'b', 'c'` num **único argumento** (apesar das vírgulas internas). Dentro da macro, `%1` é essa lista inteira, que é despejada no `db`.
- No expandido (linhas 221–225):

```

..@26.str: db "11. Sub-lista passou: ", 'a', 'b', 'c', 10

```

As chaves somem; os três caracteres entram como elementos do `db`.

12. Macro sem parâmetros + `.nolist` — `macro_zero_nolist`

- **Fonte:** `%macro macro_zero_nolist 0.nolist`.
- **Chamada:** `macro_zero_nolist` (linha 197).
- **Substituição:** `0` = nenhum parâmetro. O `.nolist` **suprime a expansão no arquivo de listagem** (`-1`), mas o corpo ainda é gerado normalmente no código.
- No expandido (linhas 235–240): aparece o `print msg_zero, len_zero` expandido como qualquer outro. O `.nolist` só afeta o `.lst`, não o `-E`.

13. Concatenação a dígito `%{1}1` — `concat_digito`

- **Chamada:** `concat_digito A` (linha 200).
- **Substituição:** `var%{1}1` usa **chaves para delimitar** o parâmetro antes de colar o dígito `1`. Sem as chaves, `%11` seria interpretado como "parâmetro 11". Com `%{1}` o NASM entende "parâmetro 1" e cola o `1` literal → `varA1`.
- No expandido (linhas 244–249): bloco `print` usando `varA1` / `len_varA1`.

3. Resumo dos mecanismos do pré-processador observados

#	Recurso	Diretiva	Efeito na expansão
1	Macro básica	<code>%macro</code>	Corpo copiado, <code>%1</code> / <code>%2</code> substituídos
2	Case-insensitive	<code>%imacro</code>	Nome casa ignorando maiúsc./minúsc.
3	Repetição	<code>%rep</code> / <code>%endrep</code>	Laço desenrolado em tempo de compilação
4	Condicional de texto	<code>%ifidn</code> / <code>%else</code>	Ramo falso eliminado da saída
5	Labels locais	<code>%%label</code> , <code>%if</code>	Renomeados p/ <code>..@N.label</code> (únicos)
6	Parâmetro guloso	<code>1+</code>	Último param absorve o resto
7	Valor default	<code>1-2 50</code>	Param omitido recebe default
8	Variádico + rotação	<code>1-*</code> , <code>%0</code> , <code>%rotate</code>	Itera nº arbitrário de args
9	Concatenação	<code>var%1</code>	Cola param a identificador
10	Código de condição	<code>j%+1</code>	Concatena prefixo + condição
11	Sub-lista	<code>{ }</code>	Vírgulas internas = 1 argumento
12	Sem params / nolist	<code>0.nolist</code>	Suprime só a listagem <code>.lst</code>
13	Concat. a dígito	<code>%{1}1</code>	Chaves delimitam o índice do param

Conclusão: todas as substituições ocorrem em **tempo de pré-processamento** (antes da montagem). Condicionais e laços não existem no binário final — são resolvidos pelo NASM, gerando código já "achatado". Por isso o `_start` expandido é uma sequência linear de `mov` / `syscall` / `push` / `pop` , sem nenhuma diretiva `%`.